Section 1: Participant Information

Q1. Gender *(single choice)*

- ○ Male
- ○ Female

Q2. Current Role *(single choice)*

- ○ Backend Developer
- ○ Frontend Developer
- ○ Full-stack Developer
- ○ Mobile App Developer
- ○ DevOps Engineer
- ○ Data Engineer / ML Engineer
- ○ QA / Test Engineer
- ○ Software Architect
- ○ Other (please specify)

Q3. Years of Software Development Experience *(single choice)*

- ○ Less than 1 year
- ○ 1–2 years
- ○ 3–5 years
- ○ 6–10 years
- ○ 10+ years

Q4. Which programming languages do you use most frequently? *(multiple choice, select all that apply)*

- ☐ JavaScript / TypeScript
- ☐ Python
- ☐ Java
- ☐ C#
- ☐ Go
- ☐ C / C++
- ☐ PHP
- ☐ Rust
- ☐ Kotlin
- ☐ Other (please specify)

Section 2: AI Tool Usage

Q5. Do you use AI tools for software development? *(single choice)*

- ○ Yes
- ○ No

Q6. Which AI tools or agents do you use regularly? *(multiple choice)*

- ☐ OpenAI ChatGPT

- ☐ GitHub Copilot
- ☐ Google Gemini
- ☐ Anthropic Claude
- ☐ Cursor AI
- ☐ Perplexity AI
- ☐ Other (please specify)

Q7. How often do you use AI tools while developing software? *(single choice)*

- ○ Multiple times a day
- ○ Once a day
- ○ Several times a week
- ○ Several times a month
- ○ Rarely
- ○ Never

Q8. For what purposes do you use AI tools? *(multiple choice, select all that apply)*

- ☐ Code generation
- ☐ Bug fixing / debugging
- ☐ Explaining code
- ☐ Learning new technologies
- ☐ Writing documentation
- ☐ Writing test cases
- ☐ Refactoring code
- ☐ Database query generation
- ☐ DevOps / scripting tasks
- ☐ UI/UX generation
- ☐ Brainstorming ideas
- ☐ Research
- ☐ Other (please specify)

Section 3: Effectiveness & Productivity

Q9. How helpful are AI tools in your daily development work? *(single choice)*

- ○ Very helpful
- ○ Helpful
- ○ Neutral
- ○ Slightly helpful
- ○ Not helpful at all

Q10. How much do you trust AI-generated code without modification? *(single choice)*

- ○ Completely trust
- ○ Mostly trust
- ○ Neutral
- ○ Slightly trust
- ○ No trust at all

Q11. How often do you verify AI-generated code before using it? *(single choice)*

- ○ Always
- ○ Often
- ○ Sometimes
- ○ Rarely
- ○ Never

Q12. Have AI tools helped you learn new concepts or technologies? *(single choice)*

- ○ Significantly
- ○ Somewhat
- ○ Rarely
- ○ Never

Section 4: AI-Generated Debt in Software Development

Technical debt refers to the long-term cost, complexity, and maintenance burden that result from quick, incomplete, or suboptimal software development decisions. Please answer the following questions based on your experience with AI-assisted software development.

A. Architecture Debt

Architecture debt refers to long-term structural or scalability issues in software systems caused by poor architectural decisions.

Example: A team builds a tightly coupled monolithic application for an e-commerce platform with user accounts, product search, checkout, and inventory management. As the product grows, adding new features becomes difficult because small changes in one module often break unrelated parts of the system.

Q13. AI-generated solutions may negatively affect long-term system architecture. *(single choice)*

- ○ Strongly Agree
- ○ Agree
- ○ Neutral
- ○ Disagree
- ○ Strongly Disagree

Q14. Which architecture-related issues have you experienced from AI-generated code? *(multiple choice)*

- ☐ Poor scalability
- ☐ Tight coupling between components
- ☐ Inconsistent architecture patterns
- ☐ Difficult system integration
- ☐ Poor modularity
- ☐ Maintainability issues
- ☐ None
- ☐ Other (please specify)

B. Code Debt

Code debt refers to low-quality, overly complex, duplicated, or difficult-to-maintain source code that increases the effort required to understand, modify, or extend a software system.

Example: A developer copies the same user registration validation logic into multiple files, such as the sign-up page, admin user creation form, and profile update feature, instead of creating a shared function. Later, when the validation rules change, the team must update the same logic in several places, increasing the risk of inconsistent behavior or missed updates.

Q15. AI-generated code often increases difficult-to-maintain or low-quality code. *(single choice)*

- ○ Strongly Agree
- ○ Agree
- ○ Neutral
- ○ Disagree
- ○ Strongly Disagree

Q16. Which code-related problems have you experienced from AI-generated code? *(multiple choice)*

- ☐ Duplicated code
- ☐ Poor readability
- ☐ Overly complex implementations
- ☐ Non-standard coding practices
- ☐ Hard-to-maintain code
- ☐ Excessive unused code
- ☐ None
- ☐ Other (please specify)

C. Configuration Debt

Problems related to configuration management, dependencies, environments, or deployment setup.

Example: Assume a customer support dashboard relies on manually configured environment variables and undocumented dependency versions; when a new developer sets it up or the team deploys it, small configuration differences cause unexpected errors.

Q17. AI-generated solutions may introduce configuration or dependency management issues. *(single choice)*

- ○ Strongly Agree
- ○ Agree
- ○ Neutral
- ○ Disagree
- ○ Strongly Disagree

Q18. Which configuration-related issues have you noticed? *(multiple choicey)*

- ☐ Incorrect environment configurations
- ☐ Version conflicts
- ☐ Deprecated libraries / frameworks
- ☐ Unnecessary dependencies
- ☐ Misconfigured deployment settings
- ☐ Compatibility issues across environments
- ☐ None
- ☐ Other (please specify)

D. Design Debt

Design debt refers to poor software design decisions, weak abstractions, temporary workarounds, or misplaced logic that make a system harder to understand, modify, or extend.

Example: Assume a developer is building a checkout page and places the discount calculation logic directly within the page to quickly complete the shopping cart feature. Later, when the same discount logic is needed for invoices and order summaries, the team must either duplicate the logic or refactor the system, increasing maintenance effort.

Q19. AI tools sometimes suggest solutions that conflict with good software design principles. *(single choice)*

- ○ Strongly Agree
- ○ Agree
- ○ Neutral
- ○ Disagree
- ○ Strongly Disagree

Q20. Which design-related issues have you noticed in AI-generated code? *(multiple choice)*

- ☐ Poor separation of concerns
- ☐ Temporary workaround implementations
- ☐ Overly long functions / classes
- ☐ Repeated business logic
- ☐ Weak abstractions or design patterns
- ☐ Poor reusability
- ☐ None
- ☐ Other (please specify)

E. Documentation Debt

Documentation debt refers to missing, incomplete, outdated, or poor-quality documentation that makes a software system harder to understand, use, maintain, or extend.

Example: Assume a team is updating the refund processing API in an e-commerce application to require a refund reason and transaction ID, but does not update the related documentation. Later, developers continue using the old API documentation, which leads to failed refund requests, wasted debugging time, and confusion about the correct refund workflow.

Q21. AI-assisted development may lead to incomplete or low-quality documentation. *(single choice)*

- ○ Strongly Agree
- ○ Agree
- ○ Neutral
- ○ Disagree
- ○ Strongly Disagree

Q22. What documentation problems have you experienced with AI-generated content? *(multiple choice)*

- ☐ Missing documentation
- ☐ Outdated explanations

- ☐ Incorrect comments
- ☐ Generic / non-useful documentation
- ☐ Inconsistent documentation
- ☐ Hallucinated or inaccurate information
- ☐ None
- ☐ Other (please specify)

F. Infrastructure Debt

Infrastructure debt refers to issues related to deployment infrastructure, cloud services, CI/CD pipelines, monitoring, or operational environments that make a software system harder to deploy, scale, observe, or operate reliably.

Example: Assume a team is building a real-time order tracking feature that shows customers the current status of their delivery. However, the feature must be deployed manually because the CI/CD pipeline was never fully automated. Over time, updates to the tracking feature become slower and riskier, since each release depends on manual deployment steps that can be forgotten, performed inconsistently, or become difficult to troubleshoot.

Q23. AI-generated solutions may introduce infrastructure or operational inefficiencies. *(single choice)*

- ○ Strongly Agree
- ○ Agree
- ○ Neutral
- ○ Disagree
- ○ Strongly Disagree

Q24. Which infrastructure-related problems have you experienced? *(multiple choice)*

- ☐ Inefficient CI/CD configurations
- ☐ Poor monitoring / logging setup
- ☐ Cloud resource misconfiguration
- ☐ Security issues in infrastructure setup
- ☐ Inefficient deployment scripts
- ☐ Scalability or reliability problems
- ☐ None
- ☐ Other (please specify)

G. Management Debt

Project management or process-related issues that negatively affect software development and maintenance.

Example: Assume a team is building a subscription billing feature without clearly defining requirements, priorities, or ownership. Over time, developers implement overlapping billing rules, miss dependencies with the payment system, and become unclear about who is responsible for maintaining the feature.

Q25. Heavy reliance on AI tools may create management or workflow-related challenges within software teams. *(single choice)*

- ○ Strongly Agree

- ○ Agree
- ○ Neutral
- ○ Disagree
- ○ Strongly Disagree

Q26. Which management-related issues have you observed due to AI-assisted development? *(multiple choice)*

- ☐ Reduced code review quality
- ☐ Over-reliance on AI-generated solutions
- ☐ Increased technical decision confusion
- ☐ Inconsistent development practices across team members
- ☐ Reduced knowledge sharing within teams
- ☐ Unrealistic productivity expectations
- ☐ None
- ☐ Other (please specify)

H. People Debt

Human-related issues such as communication gaps, reduced collaboration, skill erosion, or onboarding difficulties.

Example: Assume a team uses AI-generated code to build a login feature with password reset and role-based access, but does not review the implementation together. Later, developers struggle to understand the authentication logic, making onboarding, debugging, and future changes harder.

Q27. AI-assisted development may negatively affect collaboration, learning, or developer skill growth. *(single choice)*

- ○ Strongly Agree
- ○ Agree
- ○ Neutral
- ○ Disagree
- ○ Strongly Disagree

Q28. Which people-related challenges have you experienced due to AI-generated code? *(multiple choice)*

- ☐ Reduced understanding of generated code
- ☐ Difficulty onboarding new developers
- ☐ Reduced collaboration among team members
- ☐ Over-dependence on AI tools
- ☐ Skill degradation / problem-solving reduction
- ☐ Communication gaps during development
- ☐ None
- ☐ Other (please specify)

I. Requirements Debt

Requirement debt refers to generated solutions that do not fully satisfy business requirements, user needs, or project goals, even if the code appears technically correct.

Example: Assume an AI tool is generating a checkout feature, but it does not include required discount rules, tax calculations, or accessibility requirements. Although the feature runs without errors, it does not meet the actual business needs and must be revised later.

Q29. AI-generated solutions may fail to fully reflect actual business requirements. *(single choice)*

- ○ Strongly Agree
- ○ Agree
- ○ Neutral
- ○ Disagree
- ○ Strongly Disagree

Q30. In which situations do AI tools fail most regarding requirements? *(multiple choice)*

- ☐ Complex business logic
- ☐ Domain-specific applications
- ☐ Client-specific customization
- ☐ Multi-step workflows
- ☐ Large-scale systems
- ☐ Ambiguous requirements
- ☐ None
- ☐ Other (please specify)

J. Test Debt

Test debt refers to insufficient, incomplete, or poorly maintained tests that make it harder to verify whether a software system works correctly and remains reliable over time.

Example: Assume a team builds a payment feature with coupon validation and refund handling, but only tests the main payment flow. Later, edge cases fail because discounts, failed payments, and refund scenarios were not properly tested.

Q31. Developers may rely on AI-generated code without proper testing. *(single choice)*

- ○ Strongly Agree
- ○ Agree
- ○ Neutral
- ○ Disagree
- ○ Strongly Disagree

Q32. How do you usually validate AI-generated code before production use? *(multiple choice)*

- ☐ Manual code review
- ☐ Writing additional tests
- ☐ Running automated tests
- ☐ Cross-checking documentation
- ☐ Rewriting parts manually
- ☐ Peer review
- ☐ Rarely validate
- ☐ None
- ☐ Other (please specify)

Section 5: Open-Ended Questions

Q33. Describe a situation where AI tools helped reduce technical debt and improved software quality in your development work. *(open-ended)*

Guidance shown to respondents: A strong response may describe the situation, explain how the AI tool helped, and discuss how it improved code quality, maintainability, testing, documentation, collaboration, or future development.

Q34. Describe a situation where AI-generated code or AI-assisted development introduced technical debt or caused problems in your software project. *(open-ended)*

Guidance shown to respondents: A strong response may describe the situation, explain what type of problem occurred, and discuss how it affected development, maintenance, collaboration, testing, or future changes.

Q35. What improvements would you like to see in AI coding assistants to reduce the risk of AI-generated technical debt? *(open-ended)*

Guidance shown to respondents: A strong response may describe one or more improvements, explain why they would be useful, and discuss how they could help reduce technical debt in software development.